Solución del taller — P3: monitoreo y alertas de conservación

Nombre de la solución: Conserva.

Institución destinataria: Corporación Luis Eduardo Nieto Arteta (CLENA), Archivo Histórico, Barranquilla.

Usuarios principales: archivistas y restauradores.

Equipo de origen: G7.

Equipo Backend: nombres y roles pendientes de suministrar.

Naturaleza de este documento: propuesta técnica desarrollada y respaldada por un prototipo funcional; validación institucional pendiente.

1. Planteamiento del problema

Según el enunciado, el estado de los documentos se revisa de forma visual, manual y esporádica. La información dispersa dificulta saber qué expediente requiere atención, cuándo se inspeccionó por última vez y quién debe intervenir. La humedad, las plagas y el desgaste pueden avanzar sin alertas oportunas, con consecuencias irreversibles para material único.

La necesidad no es reemplazar el criterio del restaurador: es registrar sus observaciones, detectar condiciones configuradas como riesgosas y facilitar una respuesta trazable. El problema pertenece al Archivo Histórico y fue identificado por G7. No se presume haber recibido fotos, entrevistas ni datos reales adicionales.

2. Objetivo general

Desarrollar una aplicación web que centralice el inventario de expedientes y su estado de conservación, registre inspecciones y condiciones ambientales, genere alertas tempranas y permita asignar intervenciones, mediante NestJS, Clean Architecture, React y PostgreSQL.

Objetivos específicos

1. Ubicar cada expediente por depósito, estantería y caja.

2. Registrar inspecciones con estado Óptimo, Regular o Crítico y conservar su historial.

3. Automatizar la detección de estado crítico, inspecciones vencidas y lecturas ambientales fuera de rango.

4. Recibir temperatura y humedad mediante una API autenticada y simular sensores sin hardware.

5. Ordenar las alertas y asignar acciones con responsable, plazo y evidencia de ejecución.

6. Proteger la información mediante roles y sesiones revocables.

7. Entregar código, pruebas, migraciones, contenedores y manuales que permitan operar el sistema.

3. Actores e interesados

Actor	Responsabilidad / necesidad
Administrador	Crear usuarios y depósitos, configurar umbrales, archivar expedientes y supervisar intervenciones
Personal CLENA	Consultar inventario, registrar inspecciones y lecturas, asignar tareas y actualizar las propias
Usuario público	Registrarse, gestionar su cuenta y consultar privacidad; no accede al inventario reservado
G7 / Santiago Herrera Arévalo	Aportar contexto, validar backlog, asistir a reviews y aprobar la entrega
Product Owner Backend	Priorizar y coordinar validaciones con G7
Scrum Master	Facilitar ceremonias y comprobar evidencia verificable
Development Team	Implementar, probar, revisar PRs y preparar operación

La distribución nominal de roles no se inventa; debe acordarla el equipo.

4. Alcance funcional

Incluido: autenticación, inventario, depósitos, inspecciones, sensores simulados, motor de reglas, dashboard, intervenciones, administración de usuarios, migraciones, semillas y documentación.

Fuera del alcance de P3: catálogo bibliográfico público, préstamos, gestión de solicitudes de consulta, digitalización masiva, almacenamiento de fotografías de expedientes, compra de sensores físicos y decisiones automáticas sobre tratamientos de restauración.

El registro de plagas y desgaste se realiza como observaciones de inspección. No se afirma que los sensores de temperatura/humedad detecten plagas ni deterioro físico.

5. Reglas de negocio

Regla	Condición	Resultado
R1. Estado crítico	Inspección en CRITICO	Alerta de prioridad 1 para el expediente
R2. Ambiente	Temperatura o humedad por debajo del mínimo o por encima del máximo del depósito	Alerta de prioridad 2 para el depósito
R3. Inspección vencida	Fecha de próxima inspección anterior a la fecha actual y expediente activo	Alerta de prioridad 3
R4. Límites incluidos	Valor igual al mínimo o máximo	No constituye salida de rango
R5. Deduplicación	Riesgo ya tiene alerta abierta	Se actualiza la misma alerta; no se duplica
R6. Recuperación	Nueva inspección no crítica o lectura vigente dentro de rango	Resuelve la alerta del tipo correspondiente y conserva el historial
R7. Lectura atrasada	La medición es anterior a la vigente	Se guarda, pero no reemplaza el estado ambiental vigente

Regla	Condición	Resultado
R8. Tarea finalizada	Se describe una intervención de al menos 10 caracteres	Tarea completada; el riesgo requiere comprobación posterior
R9. Primera revisión	Nuevo expediente sin inspección	Queda pendiente desde su creación; la evaluación periódica detecta el vencimiento
R10. Archivo lógico	Administrador retira un expediente	Se preservan inspecciones y tareas; se resuelven alertas del expediente y no se aceptan nuevas inspecciones

La periodicidad permitida es de 1 a 365 días. El personal define el valor adecuado. El motor de vencimientos se ejecuta al iniciar y cada 60 segundos, además del botón de evaluación manual. No se prometen notificaciones por SMS ni correo: el alcance de alertas implementado es el dashboard.

Los datos semilla usan 18–24 °C y 40–60 % de humedad relativa **solo para demostrar el funcionamiento**. La definición de rangos de conservación exige validación de los responsables de la colección.

6. Requisitos no funcionales

• Dominio y aplicación en TypeScript puro, sin dependencias de NestJS ni TypeORM.

• API REST `/api/v1` y OpenAPI en `/docs`.

• JWT access de 15 minutos y refresh de 7 días, rotación y revocación de sesiones.

• Hash bcrypt de contraseñas; secretos externos al código.

• Roles ADMIN, STAFF y PUBLIC verificados en backend y rutas protegidas en frontend.

• Entradas validadas y respuestas uniformes de error.

• Paginación con límite máximo de 100, filtros y columnas de ordenamiento permitidas.

• PostgreSQL con integridad referencial, transacciones, índices y migraciones versionadas.

• Interfaz adaptable a móvil, etiquetas, foco visible y navegación por teclado.

• CI que ejecute lint, pruebas y build; cobertura mínima de 70 % en capas internas.

• Arranque reproducible con Docker Compose, manual de operación y respaldo.

7. Diseño de arquitectura

Se adopta un monorepo con `backend/` y `frontend/`. La presentación recibe solicitudes, valida DTOs, autentica al usuario y llama casos de uso. La aplicación coordina las acciones. El dominio contiene tipos, reglas y puertos. La infraestructura implementa persistencia y seguridad. El módulo de NestJS ensambla las clases mediante tokens.

Ejemplo de trazabilidad de una inspección:

1. React envía `POST /api/v1/records/:id/inspections` con JWT.

2. El guard confirma sesión y rol; el DTO valida el cuerpo.

3. `RegisterInspection` abre una transacción del puerto `Repository`.

4. Se crea la inspección, actualiza estado y fecha de revisión y evalúan alertas.

5. `PgRepository` persiste todo con TypeORM en PostgreSQL.

6. Se confirma la transacción y React actualiza la ficha y su historial.

El detalle de dependencias y decisiones está en `docs/arquitectura.md`.

8. Modelo de dominio y datos

Entidades: Usuario, Sesión, Depósito, Expediente, Inspección, LecturaAmbiental, Alerta e Intervención.

Un depósito contiene muchos expedientes y lecturas. Un expediente acumula inspecciones. Una alerta pertenece a un depósito y, cuando corresponde, a un expediente. Una intervención vincula una alerta con un responsable. Los usuarios tienen sesiones revocables y actúan como inspectores o responsables.

Los códigos de expediente y correos son únicos. Existe como máximo una alerta abierta por clave de riesgo y una intervención no finalizada por alerta. Los historiales no se borran al archivar un expediente. El diagrama ER y el diccionario se encuentran en `docs/modelo-datos.md`.

9. Product Backlog y planificación

El backlog contiene historias de acceso, inventario, inspecciones, reglas, sensores, tablero, intervenciones, privacidad y operación. Cada historia tiene rol, beneficio, criterios observables y puntos. El detalle completo está en `docs/product-backlog.md`.

Propuesta de tres sprints de dos semanas:

Sprint	Objetivo	Resultado verificable
1	Asegurar acceso y organizar el inventario	JWT/roles, depósitos, expedientes, migraciones, CI y primera review
2	Detectar riesgos y mostrar información útil	Inspecciones, ingesta, simulador, alertas y dashboard integrado
3	Cerrar el flujo de intervención y preparar entrega	Tareas, privacidad, pruebas finales, hosting, manuales y aceptación

Se propone GitHub Projects con Backlog, To Do, In Progress, In Review y Done. Cada historia se convierte en Issue, se asocia a un milestone y se implementa en rama corta. La fusión requiere PR aprobado y CI verde. Las fechas reales de sprints no están especificadas en el documento; no se asignan fechas ficticias.

10. Estrategia de pruebas

Unitarias: límites ambientales, reglas de fechas, estados de tareas, inspecciones, deduplicación, lecturas atrasadas, archivo lógico, autenticación, rotación, eliminación de cuenta y protección del último administrador.

E2E contra PostgreSQL: validación de cuerpos y parámetros, roles, inventario, errores por duplicado, inspecciones concurrentes, alerta ambiental, asignación y cierre de tareas, vencimientos, archivo lógico y sesiones.

Navegador: login, dashboard, búsqueda de expediente, registro de inspección, pantalla móvil, logout y restricciones de cuenta pública.

Los resultados medidos están en `docs/verificacion.md`. La cobertura no sustituye la validación con archivistas ni demuestra por sí sola cumplimiento de accesibilidad o protección de datos.

11. Despliegue y operación

Docker Compose genera una configuración aleatoria persistente, levanta PostgreSQL 17, ejecuta migraciones y semillas e inicia los servicios. La aplicación local se consulta en `http://localhost:8080`. El backend incluye `/health`; Nginx sirve React y enruta `/api` al servicio interno.

El despliegue público requiere servidor o proveedor, dominio/URL y TLS. Es un pendiente de entrega; la instancia local no se presenta como URL pública. El procedimiento de respaldo, restauración y variables está documentado en `docs/manual-operacion.md`.

12. Privacidad, ética y mantenibilidad

Solo se incluyen datos sintéticos. No se incorpora información real de expedientes, fotografías ni datos personales del CLENA sin autorización. El consentimiento se registra, el inventario se restringe por rol y la eliminación de cuenta anonimiza identificadores personales manteniendo trazabilidad institucional.

Antes de producción deben acordarse con la institución el responsable de tratamiento, canal de atención, retención de datos, licencia y propiedad del código. El prototipo no certifica cumplimiento jurídico. Se documenta el uso de IA como asistencia; cada integrante debe entender y poder modificar el código que aporte.

13. Riesgos y mitigaciones

Riesgo	Mitigación
Rangos ambientales no validados	Configuración por depósito y validación explícita con restauradores
Alertas repetitivas	Índice único parcial y deduplicación de riesgos abiertos
Cambios concurrentes inconsistentes	Transacciones y bloqueo de aplicación en PostgreSQL
Pérdida de información	Volumen persistente, respaldo periódico y prueba de restauración
Datos reales publicados	Semillas ficticias; revisión antes de subir inventarios y evidencias
Dependencia del equipo	Docker, manuales y guion de operación
Evidencias Scrum incompletas	Registrar sesiones reales en Teams y vincular Issues/PRs y videos
Lecturas perdidas o sensor desconectado	El tablero muestra hora de medición; detección automática de desconexión queda como mejora futura

14. Estado de entrega y pendientes

Se entrega implementación local funcional, documentación y pruebas. No se han realizado en nombre del equipo reuniones, ceremonias, aprobaciones ni publicaciones en GitHub. Para completar la entrega académica e institucional faltan:

1. Registrar integrantes y responsables Backend.

2. Reunirse con G7 y obtener validación del backlog y umbrales.

3. Crear repositorio, invitar al docente, proteger main y registrar Issues/Projects/PRs reales.

4. Ejecutar y grabar las ceremonias, con aportes de todos los integrantes.

5. Acordar licencia, privacidad y operación con CLENA y universidad.

6. Publicar en una URL accesible con TLS.

7. Realizar demo final, pruebas de aceptación y firma del acta por G7.

15. Fuentes

• Enunciado proporcionado: «Proyecto Final Backend — Soluciones para el CLENA», apartados 3 (P3), 5–10.

• Repositorio del equipo de origen indicado en el enunciado:
https://github.com/jjparraroa-lab/proyect_into_ingsistemas.

• NestJS, documentación oficial: https://docs.nestjs.com/.

• TypeORM, migraciones: https://typeorm.io/docs/migrations/why/.

Las prioridades, reglas detalladas y organización del prototipo son decisiones propuestas para validar; no se atribuyen a una entrevista que no se ha realizado.